

GENERATIVE SYNTH ENGINE

Logan Kuzyk
University of Victoria
logan@kuzyk.net

Tai Kilpatrick
University of Victoria
taiharrison1@gmail.com

Ronen Somogyi-Csizmazia
University of Victoria
rsomogyi02@gmail.com

ABSTRACT

Our project explored the application of deep unsupervised training algorithms and model inference pipelines in generating musical samples. The objective was to develop a tuneable model that generates a sound from scratch based on training data, and extrapolates its own signal. Our aim was for it to be useful in the context of music production, with the end goal of providing a way for users to download and run the model locally without the need to maintain a public web service. We identified an opportunity to simplify the process of sampling and make it more accessible to less experienced users.

1. INTRODUCTION

This model would serve as a standalone instrument or as a foundation upon which further musical applications could be built.

This project builds off of the latent space encoding from Generating Sound with Neural Networks [5] with the usage of spectrograms as opposed to raw audio signals, inspired by MelNet [1].

Previous research laid the groundwork for efficient and accurate audio generation, but with the intention of applying to non-musical fields like speech generation. This project positions itself as an art-adjacent application and could be considered a novel synthesizer engine.

Despite our academic experience in machine learning theory, we were first-time developers in the field. Therefore, research and experimentation were major aspects of the development process. Some of our key challenges included applying techniques described in research papers and ensuring high-quality and interesting output audio. Through this project, we aimed to build our skills in machine learning and data science while developing a valuable tool for music production.

2. METHODOLOGY

Data Collection and Preprocessing — To begin the project, we gathered a diverse dataset of instrument recordings from publicly available sources and established li-

braries. The audio data was carefully preprocessed into a format suitable for both the variational autoencoder (VAE) and the subsequent recurrent neural network (RNN) stages. This preprocessing involved segmenting the audio into manageable pieces, normalizing amplitude levels for consistency, and converting the recordings into time–frequency representations as spectrograms. Additionally, each audio sample was tagged with relevant metadata, including instrument labels, to facilitate conditioning during the synthesis process. We did the training using the N-Synth [6] and the MAESTRO dataset [7].

Variational Autoencoder — In the initial phase of the project, we implemented a VAE with the primary goal of learning compact, latent representations of instrument sounds. This model was designed to capture the underlying structure of various instrument timbres, enabling it to represent complex audio characteristics in a lower-dimensional latent space. The VAE was trained to reconstruct the original audio from these latent representations, ensuring that the learned latent space accurately encapsulated the essential features of the instrument sounds. This approach not only allowed us to evaluate the quality of the reconstruction but also provided insights into how different instrument sounds relate to each other within the latent space, which was critical for subsequent stages of the project.

Model Development - Once have established a robust latent space using the VAE, we transitioned to a sequential model by evolving the architecture into an RNN-based synthesis system. In this phase, the latent representations produced by the VAE will serve as inputs to the RNN, which will be responsible for generating coherent sequences over time. We employed Long-Short-Term Memory (LSTM) layers to capture the temporal dynamics of the audio. The RNN will be designed to integrate conditioning information, including instrument labels, to ensure that the generated sequences reflect the desired timbral characteristics. The final synthesis pipeline will involve the RNN generating a sequence in the latent space, which will then be decoded back into an audio waveform using the VAE's decoder or an alternative vocoder.

Model Training - The training process for this project will be iterative and adaptive. We will begin by training the VAE with a combination of reconstruction loss, such as Mean Squared Error, and a KL-divergence term to ensure that the latent space is well-structured and meaningful. After achieving a stable VAE, the focus will shift to fine-tuning the RNN using sequences derived from the



VAE’s latent representations. Throughout both stages, we will systematically tune hyperparameters such as learning rates, batch sizes, and sequence lengths to optimize model performance.

Synthesis and Evaluation - Once preliminary models are in place, we will develop an inference pipeline to generate audio from given melodies, conditioned on chosen instrument characteristics. We plan to conduct listening tests to assess the realism and expressiveness of the generated instrument sounds. In parallel, objective metrics such as reconstruction loss, spectral convergence, will be used to quantitatively evaluate performance. Feedback from these assessments will be used to inform further adjustments to the model architecture and training procedures.

Iterative Process - Given the exploratory nature of this project, the methodology was adapted over time. Early experimental results informed subsequent modifications, ensuring that the approach remained flexible and responsive to both challenges and new insights. This evolving process was central to our strategy; we first fully developed the VAE and later branched into the RNN for more realistic sound production. Tools used for research and development included, but were not limited to, PyTorch [11] and Keras

3. TIMELINE

February — After receiving project approval, we began work by setting up the initial development and training pipelines. We focused initially on creating a variational autoencoder (VAE), which encoded the training audio signals into a latent space. From there, we trained the decoder to reconstruct audio signals in the form of Mel-Spectrograms from the latent space, and eventually converting them into audio.

March — With a functioning autoencoder, we began experimenting with ways to extend its functionality. In the earlier stages, it took awhile to adjust the tuning to generate something beyond static. By this point, we had sufficient data to train an initial proof-of-concept model and entered a cycle of testing and iterating on the project. After multiple iterations of testing and retraining the model, the model was able to take in the training frequencies, and recreate a similar sounding instrument at this frequency. We then began adding parameters to tune the model’s output. This required additional data, training, and testing.

April — By this stage, we had finished collecting training data and focused on designing a "playground" for the audio generation, where users would be able to interact with the system and select different instrument types. In parallel, we were adjusting the latent space dimensions and training parameters to best represent the different instrument types.

4. ROLES

- **Ronen:** Research and architecture design
- **Tai:** Data acquisition and pre-processing

- **Logan:** Model implementation and training

5. FINAL REPORT & RESULTS

Data Collection and Preprocessing — After the initial proposal, we have assembled a diverse dataset of instrument recordings from publicly available sources and established libraries, including the N-Synth [6], and MAESTRO [7] datasets. The audio data is preprocessed using a pipeline built in Python with libraries such as `torchaudio`, `soundfile`, and `numpy`. Each audio file is first resampled to a consistent sample rate of 16kHz and converted to mono if necessary. We then segment the audio into manageable pieces and transform these segments into time–frequency representations using **mel spectrograms**. The mel spectrograms are computed with `torchaudio`’s `MelSpectrogram` transform, providing a perceptually relevant representation that better preserves the harmonic and timbral characteristics of the instrument sounds. Each spectrogram is normalized for amplitude consistency and tagged with metadata, including instrument labels, to support conditioning in later synthesis stages.

Variational Autoencoder (VAE) — In the initial phase, we developed a regular Autoencoder then upgraded to a Variational Autoencoder (VAE) that operates on mel spectrogram frames. The VAE is implemented in PyTorch using its `nn.Module` framework, and it comprises an encoder and a decoder built from fully connected layers. The encoder compresses the input mel spectrogram (with dimensions matching the number of mel bins) into a low-dimensional latent space, capturing the underlying structure and timbral characteristics of the instruments. The decoder reconstructs the mel spectrogram from this latent representation. Training the VAE involves optimizing a composite loss function that combines Mean Squared Error (MSE) with a KL-divergence term to ensure a well-structured and meaningful latent space. Our experiments have confirmed that the VAE can capture essential features of instrument sounds, allowing for a generation of new sounds matching in timbres.



Figure 1. Figure 1. Autoencoder with no KL divergence term in loss calculation



Figure 2. Figure 2. VAE with KL divergence term in loss calculation

Initial testing from adding the Kullback Leibler divergence (Figure 2.) to the loss calculation clearly show the point neighbourhoods are more clearly defined and separated

VAE-GAN Integration Building on the robust latent representations learned by the VAE, we are integrating a Generative Adversarial Network (GAN) into our pipeline to enhance the perceptual quality of the synthesized outputs. A GAN consists of two competing neural networks, a **generator** and a **discriminator**, where the generator attempts to generate audio data, and the discriminator is trained to distinguish between real data and the data from the generator. In our hybrid VAE-GAN framework, the VAE decoder is repurposed as the generator, while a separately designed discriminator network evaluates the authenticity of the generated mel spectrograms. The discriminator is implemented as a convolutional neural network (CNN) using PyTorch’s `nn.Conv2d` and related modules to capture high-level features and ensure that the generated spectrograms are indistinguishable from real ones. The generator is trained with a combined loss function that includes reconstruction loss, KL divergence, and an adversarial loss (based on binary cross-entropy) from the discriminator. This adversarial training pushes the generator to produce more realistic and detailed spectrograms, thus improving the overall quality of the audio synthesis. We completed work in parallel to evaluate the effectiveness of the GAN, where one model was trained just using the VAE pipeline, and included the GAN corrections.

Sequential Modeling — Once a stable latent space is achieved with the VAE-GAN, we worked in parallel to determine the efficacy of the GAN design, where we conducted separate trainings with and without the GAN integration, and visually analyzed the generated latent space of the different models. The main purpose of the GAN from the adversarial loss provided by the GAN incentivizes the decoder to generate spectrograms that not only reconstruct the input accurately but also resemble real data in their distribution, thereby reducing artifacts such as blurriness and loss of detail. We compared the mel spectrograms produced by the decoder along with the generated latent space.

Sharper and More Detailed Spectrograms — Traditional VAE decoders trained solely with reconstruction losses (e.g., L1 or L2) tend to produce smooth, averaged outputs. The GAN encourages the generation of sharper spectrograms by penalizing unrealistic features, leading to finer resolution and more defined high-frequency content.

Improved Perceptual Quality — While reconstruction losses focus on minimizing pixel-wise differences, the GAN evaluates outputs based on learned, higher-level distributional features. This results in outputs that better align with human auditory perception, even if they deviate slightly at the pixel level.

Regularization of the Decoder — By requiring the decoder to “fool” the discriminator, the model learns to match the statistical properties of the training distribution beyond what a reconstruction loss alone can achieve. This provides a form of regularization that encourages more naturalistic outputs.

Foundation for Stylization and Control — The discriminator’s ability to learn rich feature representations opens the door for future extensions such as style transfer, conditional generation, and domain adaptation. These capabilities could enable the synthesis of audio in specific timbral styles or the adaptation of generated content to different musical contexts.

Model Training and Hyperparameter Tuning — Our training process has been iterative and adaptive. We initially trained the VAE independently using reconstruction loss (MSE) combined with KL divergence. With the integration of the GAN, the training regime evolved to alternate between updating the generator (VAE decoder) and the discriminator. We systematically tune hyperparameters such as learning rates, batch sizes, and the weighting factors for reconstruction, KL divergence, and adversarial losses. This joint training strategy is critical for achieving stable adversarial dynamics and ensuring that the latent space remains semantically rich.

KL Annealing Schedule — Similar to the annealing of glass or metal which is allowed to cool slowly to release internal stresses, KL annealing allows the reconstruction loss to take priority at the start of training. This allows the backpropagation algorithm to prioritize accurate reconstruction over latent space distribution[15]. With each epoch of training, the KL divergence coefficient is gradually increased, which tightens and organizes the distribution of points in the latent space. The KL divergence coefficient is only increased over the first 30 percent of epochs, past which it stays constant.

Synthesis and Evaluation — Once preliminary models were in place, we developed an inference pipeline to generate audio according to the characteristics chosen for the instrument. This took the form of a grid representing the different instrument types, where the user would select a point on the grid which corresponds to a particular region in the latent space. We conducted listening tests to assess the realism and expressiveness of the generated instrument sounds. In parallel, objective metrics such as reconstruction loss and spectral convergence were used to quantita-



Figure 3. Generated Latent Space after 25 epochs of training. The colors correspond to the different timbres of instruments (brass, woodwind, flute, string).

tively evaluate performance. Feedback from these assessments informed further adjustments to the model architecture and training procedures. An interesting phenomenon that was observed during training is the shape of the latent space. (Figure 3.) This is a plot of 100 random samples from each instrument family passed to the encoder and plotted on the latent space. The peculiar claw shape can be explained by sampling points within each claw and decoding them into audio. It turns out that each claw represents a different octave in the latent space. We compared the latent space across various training epochs to analyze the patterns that form as the model learns throughout training, and there was a clear increase in the definition and shape of the claws as training progressed.

6. FINAL RESULTS

At the current stage of the project, with the conducted training we were able to use the VAE and VAE-GAN models to generate latent space representations of the training sounds. The resulting latent space was visualized and color-coordinated according to instrument types, and displays meaningful information on the timbral characteristics, showing that the model was capable of learning meaningful, structured representations of different instruments. The results are promising towards further training and adaptation, as the generated audio outputs exhibit noticeable variation and are perceptibly different across instrument categories, confirming that the model is capable of distinguishing key features. However, at the current state the synthesized sounds do not yet closely resemble the original instruments, suggesting that while the latent space is semantically informative, further refinement is needed to achieve realistic timbral fidelity.

These results validate the foundation of our approach and point toward promising directions for enhancing realism and expressiveness in future iterations.

7. FUTURE DIRECTIONS

As our project continues to develop, there are several promising directions we intend to explore to expand the capabilities and usability of the model.

Multi-Instrument and Timbre Blending — One of our primary goals moving forward is to enhance the model’s ability to generate a wider variety of instrument types, as with current time limitations we were only able to train the model so much. This includes not only supporting additional families of instruments (e.g., strings, woodwinds, brass, and electronic instruments) but also allowing for the blending of timbres across instruments. By enabling latent space interpolations between different instrument representations, we could produce entirely new hybrid sounds. This feature would provide a lot of flexibility with different sonic textures.

Dynamic Control and Conditioning — Another potential extension involves increasing the granularity of control available to the user during synthesis. This could include conditioning the model not only on instrument labels but also on dynamics, articulation, tempo, or pitch contour. Such control mechanisms would allow for more expressive and nuanced sound generation, better simulating the natural variations of real performances and enabling precise user-directed generation in production environments.

Integration into Digital Audio Workstations (DAWs)

— To allow more practical uses of our model, we would like to allow integration as a VST to connect to DAWs which would allow integration into a proper music production environment. This would allow musicians and producers to use the generative model as an instrument or sound design tool within their existing workflows. Ideally, this plugin would offer a simple interface using our latent space grid display for generating and manipulating sounds, as well as browsing and interpolating between different timbral presets.

User-Guided Training and Style Transfer — Another exciting direction is the incorporation of user-uploaded data for custom training or fine-tuning. Musicians could train the model on their own recordings to generate sounds that reflect their unique playing style or instrument characteristics. Additionally, we may explore style transfer mechanisms where the model learns to impose the timbral or stylistic qualities of one dataset (e.g., jazz guitar) on to another (e.g., classical piano), offering creative cross-genre synthesis options.

Expanded Dataset Curation and Diversity — As we continue development, we will seek to expand and diversify our training datasets. This includes incorporating non-Western instruments, environmental or synthetic sound sources, and performance styles that are underrepresented in existing corpora. A broader dataset would enhance the richness of the model’s latent space and better support diverse musical contexts.

These future directions align with our goal of developing a flexible, high-quality, and accessible generative sound engine that supports artistic experimentation, production workflows, and musical discovery.

8. REFERENCES

- [1] S. Vasquez and M. Lewis, "MelNet: A Generative Model for Audio in the Frequency Domain," arXiv:1906.01083v1 [eess.AS] 4 Jun 2019
- [2] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C.-Z. A. Huang, S. Dieleman, E. Elsen, J. Engel, and D. Eck, "Enabling Factorized Piano Music Modeling and Generation with the MAESTRO Dataset," arXiv:1810.12247v5 [cs.SD], 17 Jan 2019. Published as a conference paper at ICLR 2019.
- [3] R. Manzelli, V. Thakkar, A. Siahkamari, and B. Kulis, "Conditioning Deep Generative Raw Audio Models for Structured Automatic Music," arXiv:1806.09905v1 [cs.SD], 26 Jun 2018. Presented at the 19th International Society for Music Information Retrieval Conference, Paris, France, 2018.
- [4] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu, "Wavenet: A Generative Model for Raw Audio," arXiv:1609.03499v2 [cs.SD], 19 Sep 2016.
- [5] V. Velardo, "Generating Sound with Neural Networks," <https://github.com/musikalkemist/generating-sound-with-neural-networks>, 2021.
- [6] D. Engemann, C. Hawthorne, R. Dieleman, and D. Eck, "NSynth Dataset," arXiv:1708.02291v1 [eess.AS], 4 Aug 2017.
- [7] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C.-Z. A. Huang, S. Dieleman, E. Elsen, J. Engel, and D. Eck, "MAESTRO: A Dataset for Music Performance Analysis and Synthesis," Google Research, 2019. [Online]. Available: <https://g.co/magenta/maestro-dataset>. [Accessed: Feb. 18, 2025].
- [8] H. Fu, C. Li, X. Liu, J. Gao, A. Celikyilmaz, and L. Carin, "Cyclical Annealing Schedule: A Simple Approach to Mitigating KL Vanishing," in *Proceedings of NAACL-HLT 2019*, Minneapolis, MN, USA, June 2–7, 2019, pp. 240–250.
- [9] TensorFlow Keras Documentation, TensorFlow, Inc., [Online]. Available: <https://www.tensorflow.org/guide/keras>. [Accessed: Feb. 18, 2025].
- [10] TensorFlow Documentation, TensorFlow, Inc., [Online]. Available: https://www.tensorflow.org/api_docs, [Accessed: Feb. 18, 2025].
- [11] PyTorch.nn Documentation, PyTorch, Facebook AI Research, [Online]. Available: <https://pytorch.org/docs/stable/nn.html>, [Accessed: Feb. 18, 2025].
- [12] S. Mehri, K. Kumar, I. Gulrajani, R. Kumar, S. Jain, J. Sotelo, A. Courville, and Y. Bengio, "SampleRNN: An Unconditional End-to-End Neural Audio Generation Model," in *Proceedings of ICLR 2017*, 2017. [Online]. Available: <https://arxiv.org/abs/1612.07837v2>, [Accessed: Feb. 18, 2025].
- [13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [14] Alex Graves. "Generating sequences with recurrent neural networks," arXiv preprint arXiv:1308.0850, 2013.
- [15] J. Shlens, "Notes on Kullback-Leibler Divergence and Likelihood Theory," arXiv:1404.2000v1 [cs.IT], Apr. 8, 2014.